```
      .onFailure[ClosedWatchServiceException](SupervisorStrategy.restart)  ❼
}

trait FileListeningAbilities {                                             ❽

  def register(uri: String) = ...
}
```

❶ Starts listening to the directory, spawns, and watches a log processor

❷ When a new file is in the directory, receives NewFile

❸ When a modified file is in the directory, receives FileModified

❹ Sends a file to the log processor

❺ Sends an event to the log processor

❻ Stop itself as the log processor will be unresumable

❼ Supervision restart. After restart, work can be duplicated; deduplication is out of the scope.

❽ Provides the file listing API

The log processor in listing 5.5 creates a database writer and monitors it as it parses files. The supervision ignores exceptions of type `ParseException` by resuming when a line in the file cannot be parsed.

When the log processor receives a termination message, it may attempt to point to another URL to re-create a database writer. This actor may find that all of its available URLs have failed and stop itself when it runs out of options.

Listing 5.5 Log processor sketch implementation

```
object LogProcessor {

  sealed trait Command
  final case class LogFile(file: File) extends Command

  def apply(): Behavior[Command] =
    Behaviors
      .supervise {
        Behaviors.setup[Command] { context =>
```